RISC-V NON-PIPELINED ARCHITECTURE

RV32I BASE INTEGER ISA

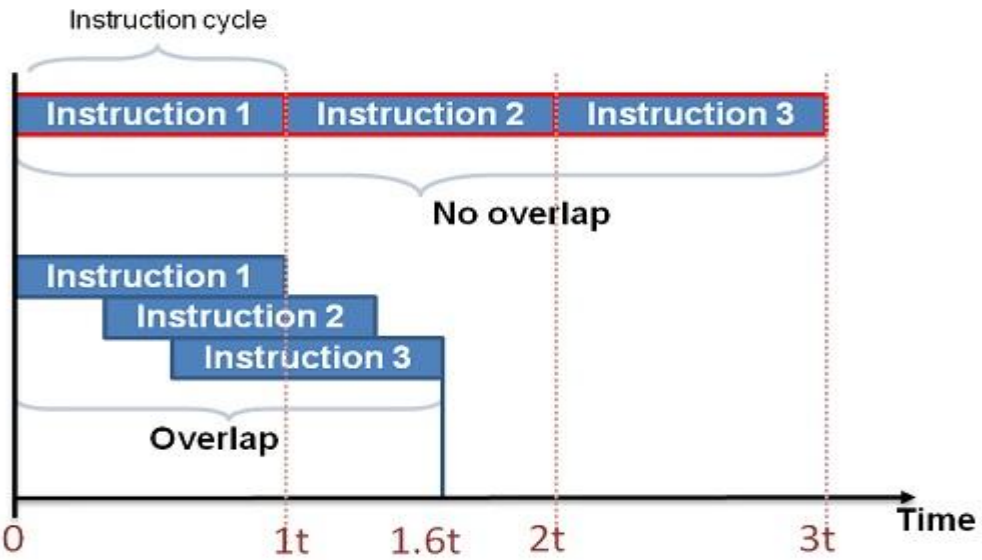
venkatasairam428@gmail.com

BY
VENKATA SAIRAM BOMMALAVARAPU

RISC-V NON-PIPELINED ARCHITECTURE

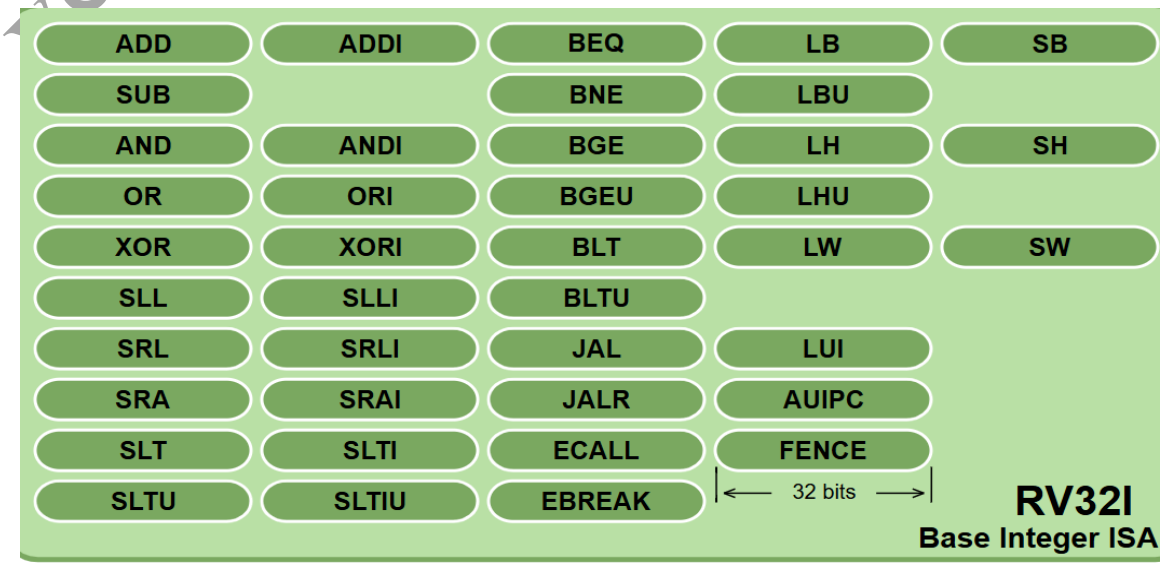
1. Introduction (Codes: https://github.com/B-V-SAIRAM428/RISC-V)

In this, I will explain how I designed an RV32I Base Integer ISA, which is a non-pipelined design (In future, I can upgrade it). Before going forward, I want to explain the difference b/w pipelined and non-pipelined. In a pipelined architecture, overlapping execution occurs, meaning multiple instructions run in parallel for better performance. In a non-pipelined architecture, execution occurs one instruction at a time. Which architecture was better and why? Pipelining was better because it will increase the speed of execution, how? Look at the figure below.



(Taken from https://witscad.com/course/computer-architecture/chapter/concepts-of-pipelining)

This architecture will contain Top_unit, which includes all the submodules like Control_unit, Instruction fetch, Instruction memory, Data_path, which contains Register, Data_memory, and ALU. We will discuss one by one. And for the testbench, I did a normal and basic verification to check whether it was working or not. In the future, I will do the verification with UVM for each module. For now, just discussed the design and basic verification, and along with that, I just checked the power and utilisation report and timing report to see if it has any validation or not. We will also discuss that.



In basic integer ISA, there are different types of instruction formats shown below. There is also another type, which is L-type, which has the same format as I-type. Each type has their own function and operation; we will go one by one.

31 27 26 25	24 20	19 15	14 12	11 7	6 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]				rd	opcode	U-type
imm[20|10:1|11|19:12]				rd	opcode	J-type

R-Type:

0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

(ADD-ADDITION, SUB-SUBTRACTION, SLL-SHIFT LEFT LOGIC, SLT-SET LESS THAN, SLTU-SET LESS THAN UNSIGNED, XOR, SRL-SHIFT RIGHT LOGIC, SRA- SHIFT RIGHT ARITHMETIC, OR, AND)

I-TYPE:

imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI

(ADDI-ADDITION, SLLI-SHIFT LEFT LOGIC, SLTI-SET LESS THAN, SLTIU-SET LESS THAN UNSIGNED, XORI, SRLI-SHIFT RIGHT LOGIC, SRAI- SHIFT RIGHT ARITHMETIC, ORI, ANDI)

L-TYPE:

imm[11:0]	rs1	000	rd	0000011	LB
imm[11:0]	rs1	001	rd	0000011	LH
imm[11:0]	rs1	010	rd	0000011	LW
imm[11:0]	rs1	100	rd	0000011	LBU
imm[11:0]	rs1	101	rd	0000011	LHU

(LB – LOAD BYTE, LH – LOAD HEX, LW – LOAD WORD, LBU - LOAD BYTE UNSIGNED, LHU – LOAD HEX UNSIGNED)

B-TYPE:

imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU

(BEQ – BRANCH EQUAL, BNE – BRANCH NOT EQUAL, BLT – BRANCH LESS THAN, BGE – BRANCH GREATER OR EQUAL, BLTU – BRANCH LESS THAN UNSIGNED, BGEU – BRANCH GREATER OR EQUAL UNSIGNED)

U-TYPE:

imm[31:12]	rd	0110111	LUI
imm[31:12]	rd	0010111	AUIPC

(LUI – LOAD UPPER IMMEDIATE, AUIPC – ADD UPPER IMMEDIATE TO PROGRAM COUNTER)

J-TYPE:

imm[20|10:1|11|19:12]			rd	1101111	JAL
imm[11:0]	rs1	000	rd	1100111	JALR

(JAL – JUMP AND LINK, JALR– JUMP AND LINK REGISTER)

S-TYPE:

imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW

(SB – STORE BYTE, SH – STORE HEX, SW – STORE WORD)

HALT:

000000000000	00000	000	00000	1110011	ECALL
000000000001	00000	000	00000	1110011	EBREAK

ECALL: When this goes high, then the PC will stop, and the system OS will take over, but in your design, we don't have any OS, so we will freeze the PC.

EBREAK: When this goes high, then the PC will stop, and the debugger will take over, but in your design, we don't have any, so we will freeze the PC.

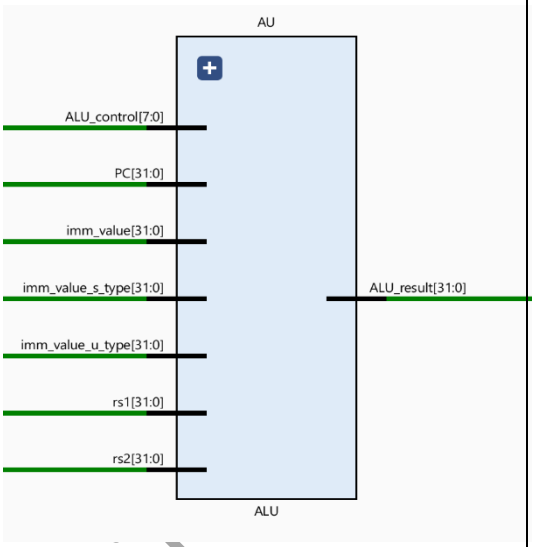
FENCE: We don't use this signal because it was used in a pipelined design for hazard, which means the possibility of data extraction will happen in the same register for two different operations, for store and load. What if the load will go for data before the store happens, so to control it, we use a fence. Anyhow, yours was non-pipeline, forgot for now.

venkatasairam428@gmail.com

2. Modules

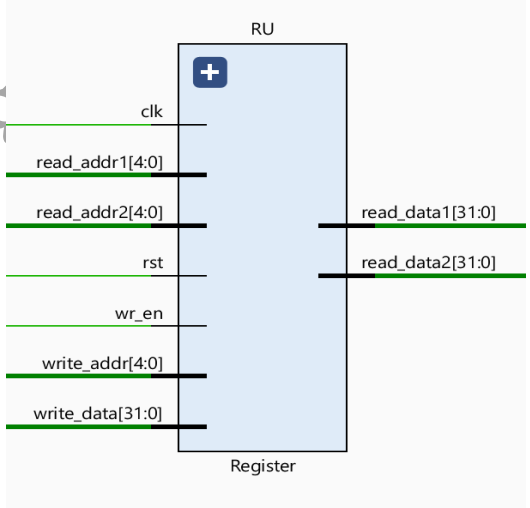
1. ALU:

It was used for different ALU operations based on ALU_control, which coming form control unit and the PC was coming from IF for the AUIPC operation, which required PC, imm_value coming from the Top_unit for I and L type operations, imm_value_s_type and imm_value_u_type are also coming from Top_unit for S and U-type operations, rs1 and rs2 data are coming from Register. ALU_result output for ALU going to different module.



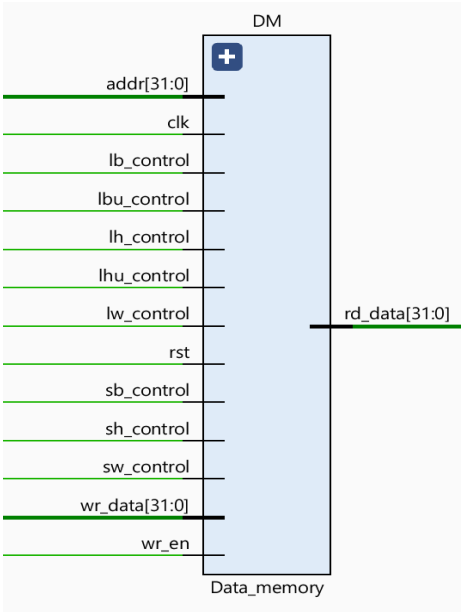
2. Register:

It will store the values and addresses in it, clk and rst are coming from Top_unit, read_addr1, read_addr2 and write_addr are coming IM output, wr_en coming from control_unit for load and store operation, when wr_en ==1 then its load and when it's low it will be a store operation, write_data was coming from the data memory and ALU_result (we will place a mux), based on read_addr1 and 2, read_data1 and 2, get the data and go to the ALU module.



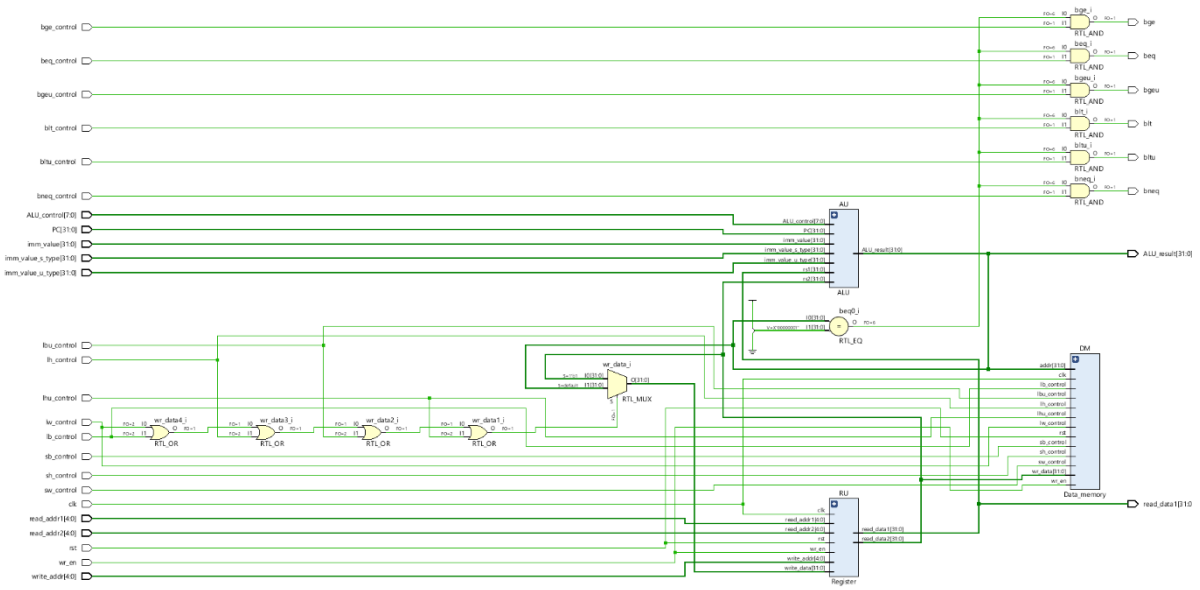
3. Data memory:

Here load and store operation will happen based on the there. controls and addr was ALU_result bcz in load operation imm_value will add to rs1 register value and form an address (all this was happen in the ALU module when a load operation) then ALU o/p ALU_result have the address that will be passed to DM addr and based on control rd_data get the data and pass to wr_data in the register, why all this mean? The load instruction means get the data from memory to the register so we need this. Now store operation, it will get the data from the register(read_data2) and store it in the data memory for that address was coming from the ALU, which is the same process like load instruction.



venkatasairam428@gmail.com

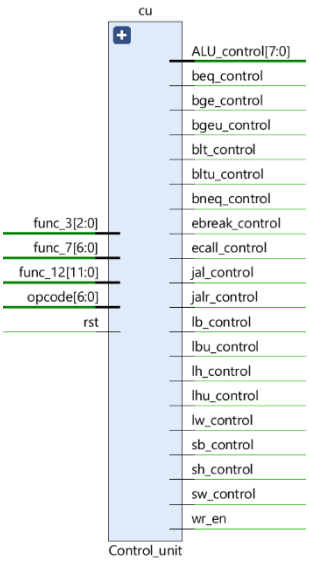
4. Datapath:



Here, all the ALU, Register and Data memory modules are placed in it, why? Just make it readable. In this module, we will assign an operation, because for a B–type operation, from the ALU we get ALU_result and from the control unit we get beq_control, bne_control …. So on. So we AND operation to make beq, bne,... so on. Go high respectively, and those signal goes to top_unit for current_pc to jump or go to the respective address. And another operation b/w load and ALU_result, why? See ALU will give a result that needs to go to the register(write_data), and when load was highs then rd_data from data memory needs to go to the register(write_data). Here is the thing: both signals are going to the same signal, so we do a mux with an assign statement in the data path and along with that, we instantiate all the remaining modules in it.

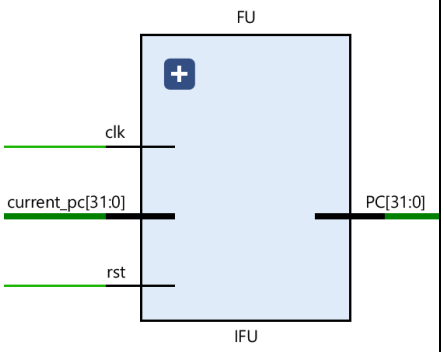
5. Control Unit:

Based on the instruction_out signal, which comes from IM, it will be passed to CU. inputs func_3(instruction_out[14:12]), func_12(instruction_out[31:20]), op_code (instruction_out[6:0]), func_7(instruction_out[31:25]), rst was coming from top top. And coming to outputs, wr_en go to data memory, B -Type controls go to data path, ALU_control to ALU and L and S type controls will go to data memory J-type control will go to the top unit, halt type also goes to the top unit.



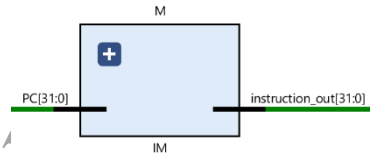
venkatasairam428@gmail.com

6. Instruction fetch:



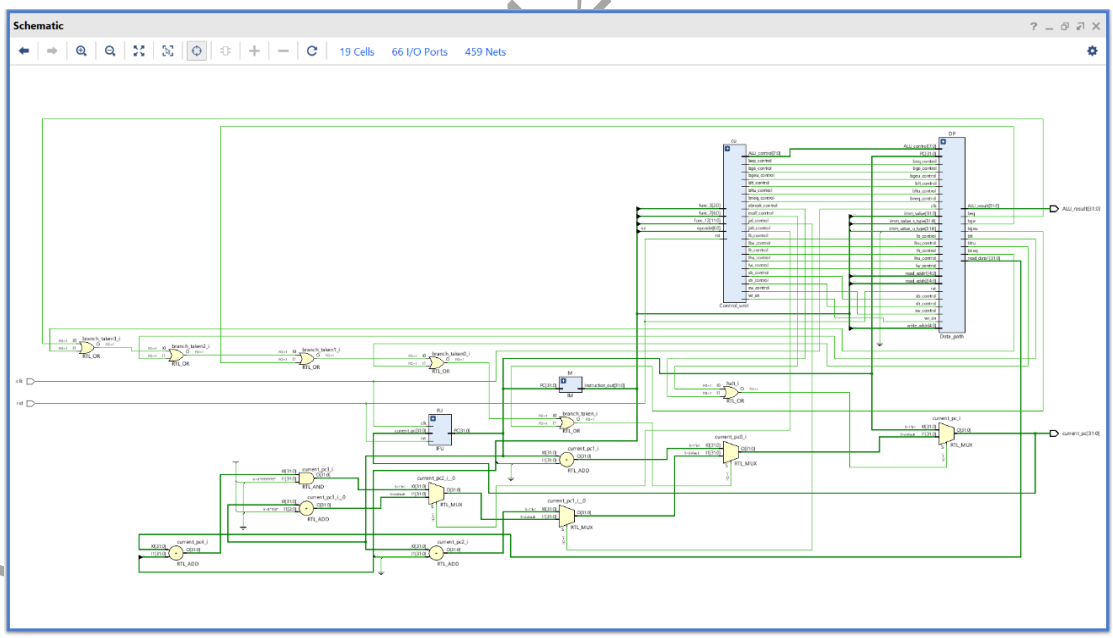
clk, rst and current_pc are coming from top_unit and in PC which the output will get the current_pc and that PC will pass to the instruction memory unit and based on that pc will get the instruction_out.

7. Instruction Memory:



It has one input PC coming from IF and one output Instruction_out which will type the operation that needs to be done. And instruction memory was written. In that initial stage, every block was empty, so we needed values, so at first, I did 4 ADDI operation with register x0, where x0 was always zero, so it will put some values in the register and then execute the remaining operation. And the store operation will be done than load will take bcz data memory was empty at the beginning, so I did the store operation.

8. Top unit:

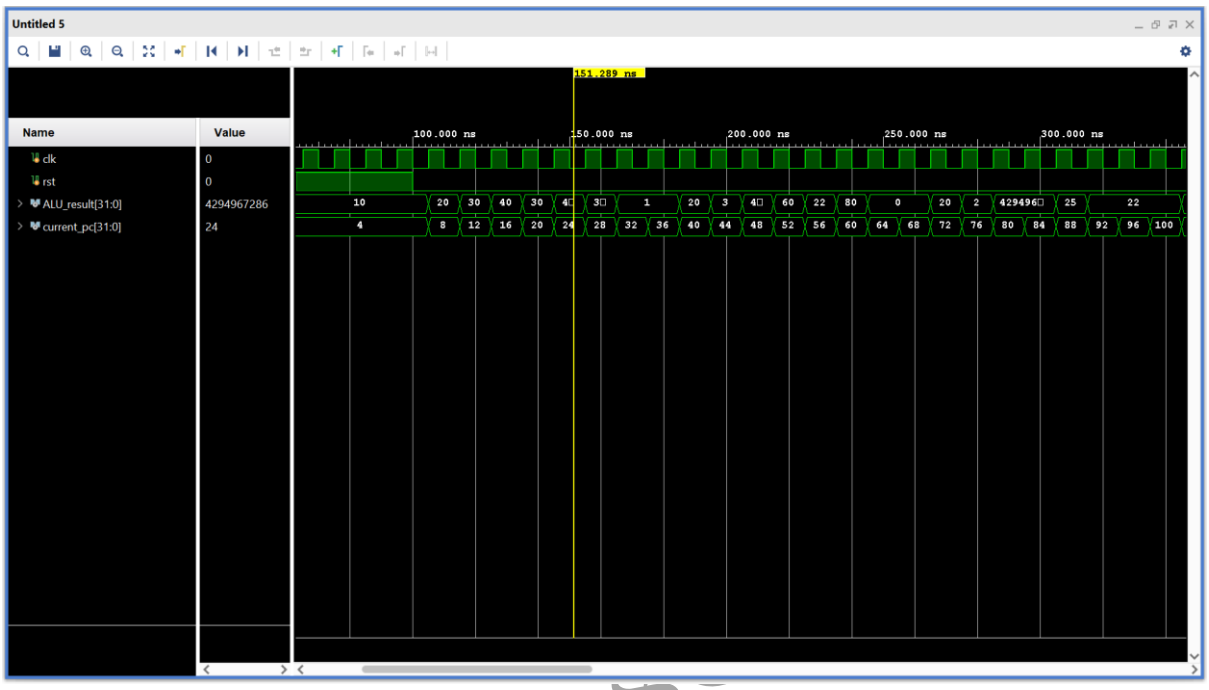


clk, rst are inputs and current_pc, ALU_result are outputs and data path, IM, IF are instantiated, and we generated a combinational circuit using assign like halt, imm_value, imm_value_u_type, imm_value_s_type, imm_addr_jal, imm_addr_jalr, imm_addr_branch and branch_taken. All this will decide the current_pc value, and that will pass to PC in IF.

Note: If you observe only the IF unit, the Register unit and the Data memory unit will have clk, because only these will need to remember the past event or need for memory, so we need FF, so clk is used.

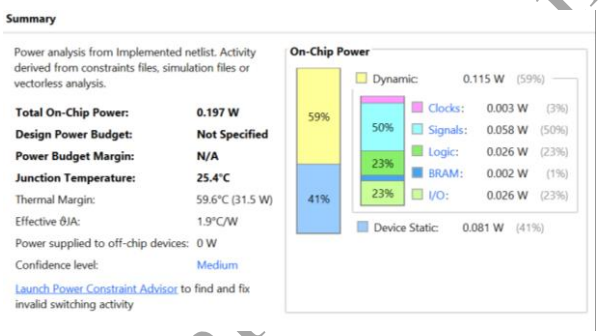
9. TB:

As I said, I did a very normal verification, in tb I just generated the clk and put the rst high for some time to check if all signals are working or not, that's it.



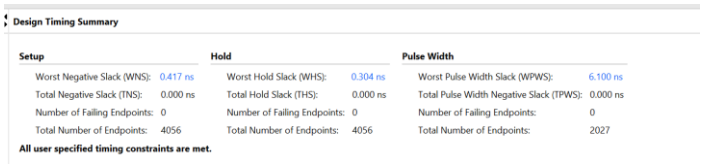
3. Analysis:

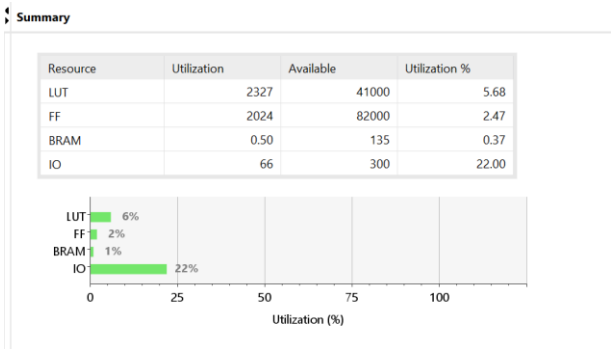
1. Power:



2. Timing:

The minimum time period was 13ns, which means fmax was 76.923MHz.



3. Utilisation Summary:

I used Kintex-7 (xc7k70tfbv676-1).